



Faculty of Mathematics and Computer Science

Multiagent systems course (MAS)

Autonomous Orchestration in Bioinformatics: Evaluating Multi-Agent Systems and LLM-Based Reasoning for Accelerated Drug Discovery Pipelines

Ciobanu Sergiu-Tudor

*Department of Computer Science, Babes-Bolyai University
1, M. Kogalniceanu Street, 400084, Cluj-Napoca, Romania
E-mail: sergiu.ciobanu@stud.ubbcluj.ro*

Abstract

The process of early-stage pharmaceutical research is traditionally hindered by extreme financial risks, high clinical failure rates, and a massive cognitive bottleneck caused by the exponential explosion of multi-omics data. While standalone Large Language Models (LLMs) and traditional deep learning scripts offer partial automation, they function as passive tools that heavily suffer from hallucinations, rigid orchestration, and a lack of interpretability. To resolve these challenges, this paper evaluates the paradigm shift toward autonomous, LLM-based multi-agent systems tailored specifically for drug discovery. We systematically analyze the foundational system design principles, memory management layers, and infrastructures that enable active software components to execute end-to-end operational tasks. Chronologically mapping the field's architectural evolution, we deeply examine framework implementations, including hierarchical supervisor configurations for automated machine learning programming and multi-perspective conversational networks utilizing frameworks like AutoGen for integrative drug-target interaction reasoning. Furthermore, we evaluate decentralized swarm-based configurations and unified orchestrator environments like the Multi-Agent Drug Discovery Orchestra (MADD) that automate customized hit identification pipelines from open-ended natural language inputs. The empirical performance evaluations across these architectures demonstrate that role-based task decomposition significantly outpaces standalone generators, achieving high accuracy, minimizing errors, and optimizing crucial structural safety constraints. Finally, we provide a critical discussion synthesizing current field limitations, such as initial human configuration bottlenecks, basic documentation libraries, and heavy dependencies on pre-curated proprietary datasets, and outline future research trajectories pointing toward automated data curation, human-in-the-loop interactions, and direct integration with physical wet-lab laboratory hardware.

© 2026 .

Keywords:

1. Introduction

1.1. Context and Problem Statement

The process of discovering and bringing a new drug to the market is notoriously difficult, expensive, and highly inefficient. Statistically, the pharmaceutical industry operates in an environment of extreme risk, where more than 80% of all therapeutic compounds that enter clinical trials end up as failures. To build a successful medicine from scratch, researchers must solve three critical problems sequentially. First, they must identify the correct biological drug target out of hundreds of thousands of gene products to determine exactly what needs to be modulated to stop complex diseases like Alzheimer's or colorectal cancer. Second, once a valid target is identified, scientists must design or discover the exact molecule that can safely interact with it. Third, they have to determine which specific patient populations will actively respond to the therapy.

At the same time, scientific research has experienced an exponential explosion of human multi-omics data. To put this data surge into perspective, the human genome was first sequenced in 2001, requiring over a decade of international collaboration and a staggering cost of approximately 11 billion dollars. Today, due to massive advancements in genomic technologies, an entire human genome can be sequenced in less than 24 hours for under 1,000 dollars. While this gives researchers access to vast quantities of biological blueprints, the sheer volume of information has created a massive cognitive bottleneck, making it impossible for human scientists to manually process, interpret, and connect all these data points efficiently.

1.2. Purpose of the Report

The main purpose of this report is to analyze how advanced computational frameworks, specifically Large Language Model (LLM) based multi-agent systems, can automate and accelerate these complex pipelines. Around 2013-2014, deep neural networks reached a level of technological potential where they began outperforming human capabilities in fundamental tasks such as image and text recognition. While the general public often views Artificial Intelligence (AI) as a magical entity capable of replacing human thoughts, its core foundation relies strictly on mathematics and computational neuroscience.

In pharmaceutical development, AI serves as an essential tool that allows researchers to investigate highly challenging biological and chemical problems with much greater confidence. By transitioning from traditional passive AI models to autonomous multi-agent systems, we can systematically lower the historical risks of drug development and improve the success rates of clinical trials. This report examines how these modern agent architectures act as digital research teams, handling end-to-end tasks from raw target prediction to molecular property validation.

1.3. Structure of the Document

The remainder of this research report is organized into distinct sections to map the progression of this paradigm shift. Section 2 establishes the theoretical and practical relevance of LLM agents, highlighting why traditional monolithic systems fall short. Section 3 outlines the core multi-agent architectures, explicitly analyzing role-based orchestration and comparing contemporary software frameworks. Section 4 focuses on cognitive reasoning workflows, diving into advanced prompting structures like the ReAct framework and the integration of heterogeneous data repositories. Section 5 presents a detailed review of experimental methodologies, validating system performance through baseline metrics and datasets. Finally, Section 6 provides a critical discussion of the current limitations, technical trade-offs, and future trajectories of agentic AI in medicine.

2. Importance and Relevance of MAS in Drug Discovery

2.1. Theoretical Significance

From a theoretical perspective, computational methods have steadily transformed chemical modeling. Traditional predictive AI tools have long been used to build Quantitative Structure-Activity Relationship (QSAR) models to

predict chemical properties, while generative algorithms have enabled the design of novel molecular profiles with custom parameters. Around 2019 and 2020, pharmaceutical companies heavily experimented with these end-to-end monolithic pipelines to generate brand-new molecular structures designed to target specific proteins.

However, these early AI implementations suffer from a critical architectural limitation: they are passive tools. They rely completely on constant human intervention to manually format datasets, feed inputs into individual scripts, interpret separate outputs, and chain the results together. Traditional pipelines cannot autonomously make decisions or handle multi-step reasoning across highly interdependent scientific workflows.

The rise of agentic AI directly addresses this bottleneck by transforming generative models into active, autonomous decision-makers. While a standalone LLM is fundamentally a passive text generator restricted to its static training dataset, an agentic framework treats the LLM as a core cognitive engine. By implementing iterative loops where an agent can systematically think, act, observe, and reflect, the system transitions from a basic chatbot to a dynamic engineering pipeline [11]. This architecture allows independent software entities to collaborate, continuously validating their own logic, modifying their internal code layouts, and correcting errors during execution.

2.2. Practical Impact

The practical integration of LLM-based multi-agent systems provides immediate operational breakthroughs by transforming how researchers handle the massive scale and multi-modal nature of pharmaceutical data. Instead of forcing a human scientist to manually spend weeks tracking down newly published academic literature or querying fragmented, specialized databases, a coordinated team of autonomous agents can rapidly retrieve papers, extract hidden biological links, and compile actionable summaries. This capability effectively removes the data-collection bottleneck that traditionally slows down the earliest phases of biotechnology pipelines.

Furthermore, multi-agent frameworks introduce a major practical shift by enabling the cross-verification of experimental leads from multiple independent angles. In a typical drug discovery setting, different computational models or data structures yield conflicting predictions. By assigning specialized agents to look at a single problem from distinct perspectives, such as calculating raw binding scores using deep learning models, identifying structural pathway relationships over biological Knowledge Graphs, and executing targeted web searches for real-world literature evidence, the system can dynamically cross-check its own findings. When these diverse viewpoints are synthesized through a central reasoning agent, the system can autonomously filter out weak hypotheses and flag critical toxicity risks, such as cardiotoxicity or liver failure, before any physical testing begins.

This selective and multi-layered verification strategy has a direct financial and clinical impact. By drastically reducing the rate of false positives in early computer-aided screening, multi-agent systems minimize the need for expensive, time-consuming laboratory wet-lab validations. Moreover, because these LLM-driven agents are instructed to execute their analytical tasks using explicit reasoning loops, they generate clear, step-by-step documentation explaining the exact rationale behind every single prediction. This systematic audit trail removes the dangerous "black-box" nature of older AI systems, providing human researchers with the clear, interpretable evidence required to satisfy strict regulatory compliance and ensure high safety margins in clinical trial design.

3. Agentic AI Design Principles and Core Architectures

3.1. Architectural Paradigms of Agents

The evolution of Artificial Intelligence in pharmaceutical research has shifted from static, monolithic models to adaptive systems known as agentic AI. Traditional approaches relied on predictive machine learning models or standalone generative tools that operated passively, requiring constant human action to structure data inputs and manually parse outputs. Agentic AI builds directly upon the foundational generation and reasoning capabilities of Large Language Models (LLMs), transforming them into active components that execute end-to-end operational tasks. While a basic standalone LLM remains isolated from the real world and is limited entirely to the information within its static training data, an agent overcomes these barriers by executing iterative loops. By connecting LLMs with programmatic infrastructure, agents can autonomously process workflows through a continuous cycle of thinking, acting, observing, and reflecting.

3.2. Core Design Principles and Tool Integration

To achieve true autonomy in biochemical research, an LLM-based agent must integrate seamlessly with specialized external components designed to perform dedicated tasks. In drug discovery workflows, these external integrations can be classified into four primary functional tool categories:

- **Perception:** These tools act as the system’s data collection layer, feeding critical information into the active LLM context. They interact with both structured and unstructured biological databases through API calls and Model Context Protocol (MCP) servers [4], querying databases such as PubChem [8], ChEMBL [19], STRING [13], and Reactome [10], and even reading the literature from PubMed.
- **Computation:** These components enable analytical modeling, property predictions, and complex simulations. They typically wrap pre-trained models like AlphaFold [7] or data processing pipelines like NextFlow [1], allowing the core LLM to select the best model based on the operational context and deploy them on local machines or high-performance computing platforms. Typical tasks include QSAR modeling [14] for bioactivity and ADMET [3], molecular dynamics simulations, and docking.
- **Action:** These tools grant the agent the capability to interact directly with physical laboratory hardware, bridging the gap between computational design and real-world execution. Through laboratory systems and robotic APIs, action tools control automated pipetting systems, next-generation sequencing library preparation, and cell-based screening assays.
- **Memory:** This layer provides persistence across multiple steps of long reasoning sessions. By utilizing vector databases like Milvus [15] or Pinecone, the agent can store, compress, retrieve, and update its active knowledge base. In pharmaceutical research, this allows the agent to retain high-value data such as Structure-Activity Relationship (SAR) patterns, standard operating procedures, and accumulated toxicity evaluations.

Together, these four pillars close the discovery loop: perception tools collect evidence to form hypotheses, computation tools turn those hypotheses into clear predictions, action tools execute physical laboratory testing, and memory tools store the rules learned from the cycle to refine subsequent iterations.

3.3. Analysis of Established System Architectures

To effectively deploy these tools, multi-agent frameworks organize their underlying LLMs into distinct structural configurations. Four primary agentic architectures have been established, each varying in coordination complexity, strengths, and ideal target tasks:

1. **ReAct Architecture:** Representing the most fundamental setup, the Reasoning-Acting paradigm blends critical thinking steps directly with dynamic tool calls. When given a task, the internal LLM decides which external tool to use and determines when to stop the loop with little to no human help. This design mimics the traditional scientific Design-Make-Test-Analyze (DMTA) cycle. It offers high flexibility and demands minimal initial setup, though it faces a risk of hallucinating tool commands. It is primarily applied to literature triage and SAR exploration.
2. **Reflection Architecture:** This architecture links multiple LLM engines together to build an environment focused on evaluation. A generator agent creates initial plans while a reflector agent reviews and critiques the logic, driving iterative improvements. This mirrors human peer review or a group of chemists discussing a project. While this setup raises overall reasoning quality, it runs slower and consumes more computing resources. It is highly effective for multi-step synthesis route planning and high-throughput workflow design.
3. **Supervisor Architecture:** This design organizes agents into a strict hierarchy modeled after a traditional corporate research team. A single supervisor agent breaks down major tasks into sub-problems, distributes them to specialized expert sub-agents, and collects their results. It excels at managing complex, multi-domain problems like autonomous HTS campaigns. However, a major bottleneck exists, suggesting that all communication must travel through the central supervisor, as sub-agents cannot talk to each other directly. This constant back-and-forth communication lengthens the conversation context, which can strain LLMs with restricted context windows.

4. **Swarm Architecture:** As a solution to the supervisor’s context bottleneck, the swarm paradigm entirely decentralizes coordination. Every peer agent connects directly to every other agent, allowing flat, distributed collaboration and direct data exchange. This approach scales exceptionally well across different laboratory sites and institutions. It is ideal for federated, multi-site toxicology data integration, though it introduces higher coordination overhead to manage the open communication loops.

3.4. Communication and Orchestration Infrastructure

To facilitate structured interactions in both supervisor and swarm frameworks, agents require open communication protocols to pass data and shared insights seamlessly. Emerging standard industrial tools—such as IBM’s Agent Communication Protocol (ACP) and Google’s Agent2Agent (A2A), which have merged under the Linux Foundation, provide a unified data exchange interface. This infrastructure enables distinct software agents to cooperate natively across separate departments, global laboratories, or independent cloud servers. In practice, this communications layer allows developers to construct highly robust platforms where a coordinator can manage complex task routing, formatting data streams into structured profiles that maintain context across a long collaborative loop.

4. Analysis of Agentic Frameworks in Drug Discovery

To understand how system design principles function in practice, we must evaluate specialized frameworks that combine multi-agent coordination with biomedical computing. This section analyzes three prominent architectures designed specifically to automate, reason through, and optimize the highly technical stages of pharmaceutical research, tracking their unique mechanics and organizational styles.

4.1. Autonomous Software Programming (*DrugAgent* - Collaboration)

The first foundational architecture focuses on automating machine learning programming within the specialized domain of therapeutics data science . Traditional engineering pipelines often fail here because translating abstract biological concepts into functional code requires deep domain expertise. General-purpose large language models routinely generate hallucinated API commands or fail to handle specialized chemical representations like SMILES strings, creating bugs that are incredibly difficult to isolate and fix .

To bridge this operational gap, the programming-focused *DrugAgent* framework [9] introduces a unified multi-agent architecture built on a hierarchical, goal-oriented supervisor setup . The system decomposes the standard machine learning workflow into a cooperative loop managed by two primary roles: an LLM Planner and an LLM Instructor . Following standard problem formulations, the system initiates its workflow from three basic parameters: a natural language *Task Description* detailing specific constraints, initial *Starter Files* such as raw compound datasets, and an *Evaluator* function that calculates performance output metrics .

The core engine of this framework operates through a highly systematic, iterative process divided between idea space management and tool execution :

1. **The LLM Planner (Idea Space Management):** Because open-ended biological tasks rarely have a single deterministic answer, the Planner is responsible for high-level strategy . In the *Idea Generation* phase, it processes the task description to derive a set of K distinct solution strategies (such as testing Graph Convolutional Networks, Random Forests, or fine-tuning pre-trained language models) . During the subsequent *Exploration* phase, the Planner selects a single strategy from the space and passes it directly to the Instructor .
2. **The LLM Instructor (Domain Knowledge and Tool Construction):** The Instructor serves as the practical programming engine . It checks the proposed plan to identify hidden domain-specific needs before launching scripts . To bypass documentation gaps or outdated library components, the Instructor references three highly targeted types of curated documentations to generate unit tests and build functional programmatic tools :
 - *Raw Data Acquisition:* Methods for securely downloading and preprocessing biological datasets from trusted open-source platforms .

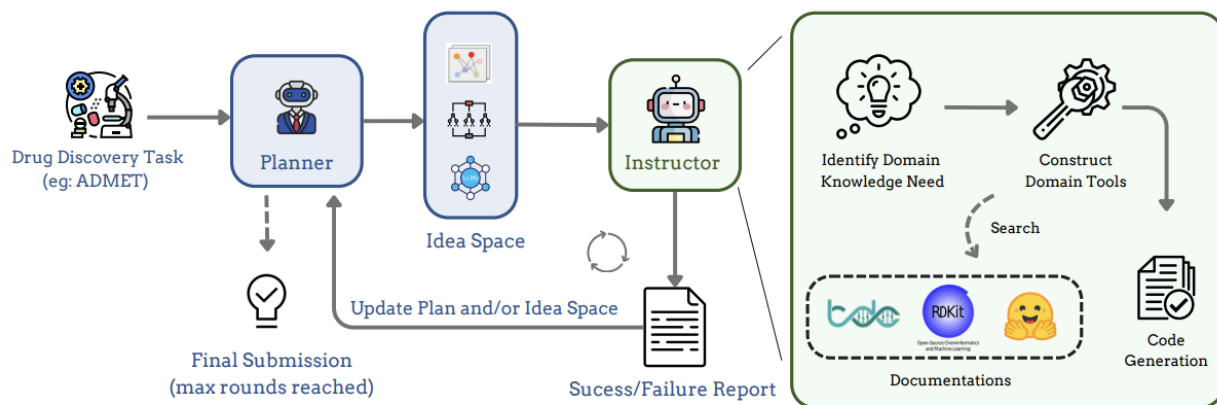


Fig. 1: The DrugAgent Architecture

- *Featurizing Biological Data*: Protocols for mapping molecular shapes into computer-readable formats, including Morgan fingerprints and graph tensors .
- *Domain-Specific Models*: Implementation guides for state-of-the-art foundation architectures, such as ChemBERTa for small molecules or Evolutionary Scale Modeling (ESM) for protein sequences .

Once the tools are built, a dedicated Coder module attempts to run the execution code . The Instructor then compiles an empirical performance report . If a critical tool or data-transformation process fails its unit testing, the Instructor feeds a detailed *Success/Failure Report* back to the Planner .

The Planner uses this real-world feedback to update its overall plan, dynamically pruning and discarding infeasible ideas while optimizing the remaining strategies . This self-correcting cycle loops continuously until a pre-set execution threshold is hit, at which point the system automatically delivers the highest-performing code script as its final submission .

4.2. Integrative Predictive Reasoning (DruGAgent - DTI Prediction)

The second major framework targets the challenge of Drug-Target Interaction (DTI) prediction, which is a critical strategy for minimizing the high costs, extended timelines, and development risks in pharmaceutical pipelines. Standalone Large Language Models struggle with accurate predictions in these advanced scenarios because their static training prevents them from analyzing complex biological mechanisms from multiple distinct perspectives. Furthermore, traditional computational methods lack the transparency and interpretability required for clinical decision-making and regulatory verification. To resolve these limitations, this branch of DruGAgent [6] introduces a conversational, multi-agent architecture modeled directly on a specialized scientific research team.

The system architecture relies on a structured, supervisor-based configuration consisting of five distinct agents, where each component is assigned a dedicated role that mirrors a specific clinical research task:

1. **Coordinator Agent**: This agent acts as the central supervisor for the entire workflow. It orchestrates the process by distributing specialized prompt tasks and managing inter-agent communication loops using the AutoGen GroupChat infrastructure [17]. To preserve data consistency across the multi-agent system, the Coordinator enforces rigid input and output data structures using Python List formatting templates. If any specialist sub-agent generates an output that deviates from these predefined parameters, the system automatically triggers an execution error.
2. **AI Agent**: This specialist focuses on deep learning data predictions. It uses the DeepPurpose package [5], specifically deploying a Message Passing Neural Network [2] combined with a Convolutional Neural Net-

work (MPNN_CNN_BindingDB), to calculate binding affinity scores directly from input molecular structures and amino acid sequences.

3. **Knowledge Graph (KG) Agent:** This agent searches through structured biomedical repositories to determine path-based relationship distances. It compiles a unified interaction table by integrating multiple curated sources, including DrugBank, the Comparative Toxicogenomics Database (CTD), STITCH, and the Drug-Gene Interaction Database (DGIdb). It calculates interaction scores based on the hop length and connection density between a drug and target node.
4. **Search Agent:** Operating in parallel, this component automates the collection of unstructured data from published medical literature. It constructs a targeted search query using the drug and target names, queries the Bing Search API to retrieve relevant titles and text, and employs an optimization engine (GPT-4o-mini) to calculate interaction weights through structural keyword matching.
5. **Reasoning Agent:** As the cognitive synthesis core, the Reasoning Agent acts on a high-tier model (OpenAI o3-mini) fine-tuned for logical tasks. It collects the normalized metrics and text summaries from all three specialists, evaluates their consistency, and blends the insights using a weighted average technique.

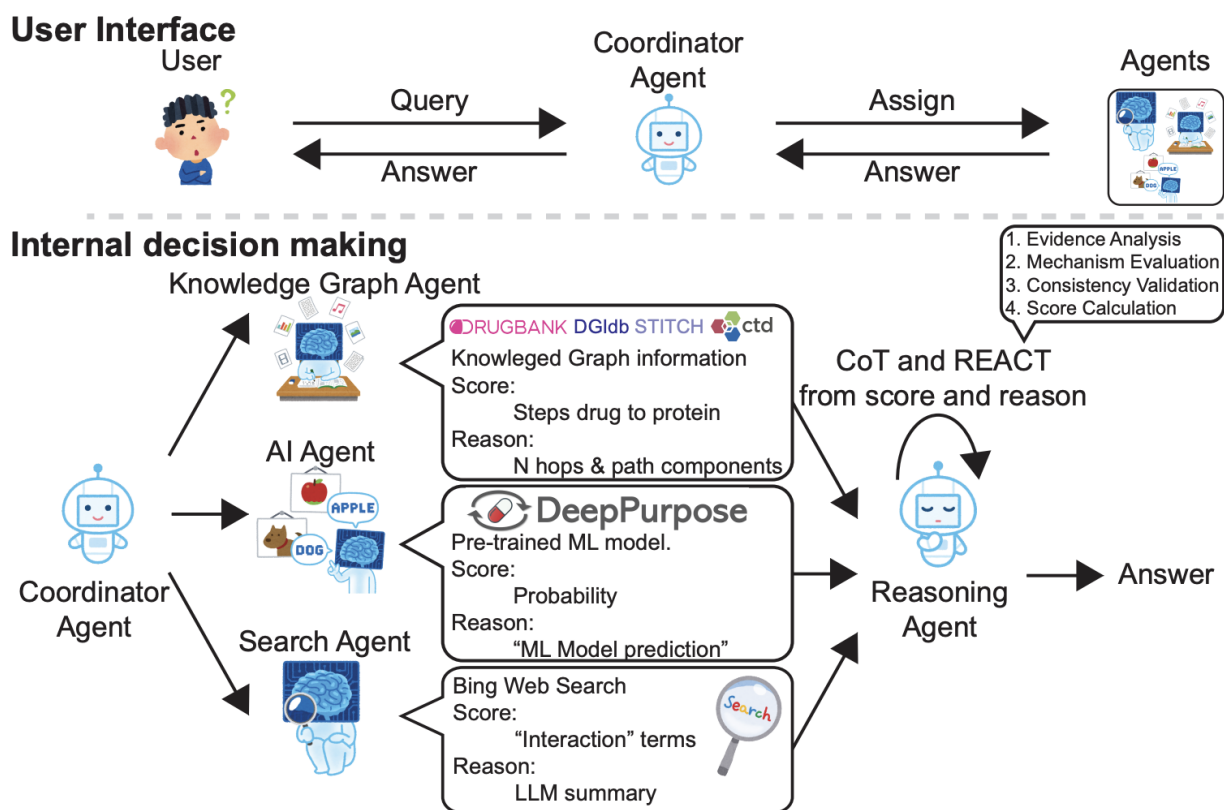


Fig. 2: The DruGAgent Architecture

A major strength of this multi-agent layout is its advanced cognitive workflow execution. While the specialized data collection agents run on lighter LLM engines to save computing resources, the central Reasoning Agent leverages the Chain-of-Thought (CoT) [16] and ReAct (Reason+Act) [18] paradigms. This setup forces the system to break down complex biochemical interactions into a step-by-step logic loop using an analytical Thought → Action → Observation pattern.

By forcing the system to explicitly document intermediate patterns, evaluate biological plausibility, and resolve conflicting signals between empirical data models and literature text, the workflow completely avoids the unveri-

fied nature of traditional neural networks. The entire pipeline ends with a structured CSV log that pairs normalized numerical probability scores with clear, step-by-step textual reasoning paths.

4.3. Scalable Multi-Agent Orchestration (MADD)

The final architectural breakthrough shifts the focus of agentic AI from descriptive data analysis to the active engineering of an end-to-end drug discovery pipeline. While predictive architectures rely on static datasets to identify candidate hits, the Multi-Agent Drug Discovery Orchestra (MADD) system [12] is designed to build and execute customized pipelines from open-ended natural language inputs. The core research hypothesis of this approach states that the distribution of distinct cognitive steps across a network of specialized agents can drastically outperform standalone language models or basic single-agent setups. By managing coordination challenges and natively embedding domain pipelines, this methodology enhances the accessibility of advanced drug design algorithms for laboratory experts.

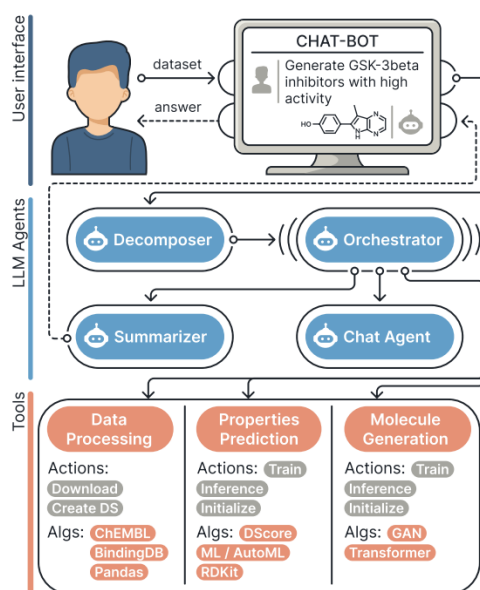


Fig. 3: The MADD Architecture

The internal configuration of MADD distributes its operational tasks among four specialized agents that interact sequentially to process complex, multi-step queries in a single execution attempt:

1. **Decomposer Agent:** When a text query is entered into the user interface, it is first routed to the Decomposer. If the objective is highly complex, this agent breaks it down into a collection of simpler, manageable subtasks. If the prompt contains ambiguities, the system calls a specialized Chat Agent to clarify instructions and collect additional context from the user.
2. **Orchestrator Agent:** This component acts as the execution heart of the framework, taking the structured subtasks from the Decomposer in sequence. Rather than trying to execute raw chemical calculations on its own, the Orchestrator constructs an action plan and coordinates a broad toolset of deep generative models and machine learning property algorithms.
3. **Summarizer Agent:** Once the computational execution finishes, the Summarizer compiles the independent data rows, calculated structural properties, and molecular shapes into a clean, structured output format to give the user an interpretable final report.

To balance task execution and protect the central agent context from overflow, MADD separates its software engineering layout by utilizing two specialized intermediate LLM-based tools, namely *DatasetBuilder* and *AutoML-DL*, to manage tool call execution instead of communicating directly with the Orchestrator. These components are split into three logical functional blocks:

- **Data Processing Layer:** Managed by the *DatasetBuilder*, this layer automatically pulls target sequences and binding information via API connections to ChEMBL and BindingDB. Users must define a target protein and an affinity metric types (such as K_i , K_d , or IC_{50}). The tool preprocesses the columns, prunes missing data fields, and balances the matrix for training.
- **Molecular Generation Algorithms:** Controlled through the *AutoML-DL*, this block includes deep generative models that build novel molecules from scratch or fine-tune architectures for specific diseases. Currently, MADD integrates an LSTM-based Generative Adversarial Network (GAN) as a fast, lightweight generator for live processing alongside a transformer-based Conditional Variational Autoencoder (CVAE) pretrained on roughly 500,000 ChEMBL compounds.
- **Property Prediction Algorithms:** This module coordinates pre-trained machine learning frameworks and features the automated pipeline builder FEDOT to train predictive models on custom dataset metrics. The system estimates the concentration needed to inhibit a biological process by 50% (IC_{50}) and calculates binding energy values through molecular docking scores. Additionally, the Orchestrator applies RDKit-based filters to track synthetic accessibility (SA), drug-likeness estimations (QED), and unwanted structural properties using PAINS (indicator of potentially false positive results in biological tests) criteria.

Through this highly organized system, MADD closes the loop between computer-aided generation and compound screening, establishing a scalable multi-agent layout for de novo pharmaceutical development.

5. Experimental Methodologies, Metrics, and Evaluation Techniques

5.1. Empirical Validation of LLM Programming Automation: DrugAgent Collaboration

To evaluate the practical capabilities of the programming-focused *DrugAgent* framework, empirical validations were conducted across three representative biomedical classification tasks: pharmacokinetic profiling via ADMET prediction (utilizing the PAMPA dataset), high-throughput screening outcome forecasting (utilizing the HIV dataset), and binding affinity determination for drug-target interactions (utilizing the DAVIS dataset). Built on the GPT-4o backend, the multi-agent system’s primary challenge was to successfully output executable machine learning scripts end-to-end without any external human troubleshooting or manual intervention.

The evaluation criteria established strict parameters for success, considering a run valid only if the generated script executed without code bugs, formatted the resulting data structures perfectly, and maintained an area under the receiver operating characteristic curve (ROC-AUC) score no more than 10% lower than a selected human expert baseline. The performance metrics were tracked over eight independent experiments and quantified based on both the final prediction accuracy and the system’s “Valid Rate” (the ratio of bug-free, passing scripts to total attempts):

- **ADMET Prediction Task (PAMPA):** Traditional single-agent structures like Chain-of-Thought (CoT) and ReAct proved highly fragile in this domain, achieving suboptimal Valid Rates of only 62.5% and 87.5% respectively due to hallucinated library modules and an inability to autonomously resolve runtime execution bugs. In contrast, the dual *Planner-Instructor* architecture of DrugAgent achieved a perfect 100% Valid Rate. Furthermore, when considering the highest-performing script among its top three structural ideas (*DrugAgent@Top3*), the system achieved a maximum ROC-AUC of 0.8206, effectively outperforming the human expert baseline score of 0.8173.
- **High-Throughput Screening Task (HIV):** On the HTS benchmark, primitive text-generation baselines continued to show critical programming vulnerabilities, with CoT collapsing to a 50% execution success rate. DrugAgent maintained its flawless 100% validation reliability across all eight code-generation cycles, utilizing

its automated unit-testing coder to fix script syntax before execution. The system achieved a solid ROC-AUC of 0.8257 under the Top-3 configuration, converging close to the human baseline of 0.8305.

- **Drug-Target Interaction Task (DAVIS):** The complexity of multi-instance sequence mapping completely broke the basic CoT paradigm, resulting in a 0% success rate (N/A). While the specialized machine learning ResearchAgent framework reached an ROC-AUC of 0.8793, it only managed a 75% Valid Rate. DrugAgent successfully stabilized code execution, recording a 100% success rate at Top-1 and a highly competitive ROC-AUC of 0.8950 under its Top-3 implementation, surpassing the baseline human expert mark of 0.8940.

These experimental results indicate that validation set optimization does not always perfectly correlate with final test set behavior in drug discovery, sometimes leading an autonomous agent to select a suboptimal idea for final script execution if limited to a single choice. However, by tracking multiple distinct ideas through its internal evaluation manager, the multi-agent system successfully mitigates this risk, yielding highly robust and generalizable machine learning tools for molecular screening.

5.2. Quantitative Results and Ablation Dynamics: Multi-Source Reasoning for DTI

The predictive performance of the conversational *DruGAgent* architecture designed for Drug-Target Interaction (DTI) analysis was evaluated using a benchmark kinase-compound activity dataset consisting of 201 proteins and 75 drugs. The system processed queries simultaneously using a ten-pair superposition method and was evaluated against a non-reasoning foundational baseline (GPT-4o mini) alongside a structural ablation study to isolate the individual contributions of each specialist node.

- **Comparison Against Non-Reasoning Baseline:** The quantitative evaluation highlights a major performance gap between the multi-source reasoning configuration and the baseline model. The default GPT-4o mini baseline followed an indiscriminate prediction strategy, labeling nearly every compound pair as interacting. While this maximized its recall metric (1.000), it resulted in a critically low precision of 0.231. Conversely, DrugAgent demonstrated a highly balanced, selective prediction strategy, reaching a precision of 0.571 and a recall of 0.476. This balanced approach yielded an F1 score of 0.514, representing a 45% relative improvement over the baseline score of 0.355. Most importantly, the reasoning model reached a specificity score of 0.978, providing a vital defense against expensive false positives in real-world validation pipelines.
- **Ablation Study Dynamics:** Systematically dropping structural components from the workflow revealed the distinct operational roles of individual agents:
 - *Removal of the AI Agent (w/o AI):* Deactivating the machine learning predictor caused the most severe performance drop across the entire system, degrading the F1 score down to 0.274, precision to 0.202, and the Area Under the Receiver Operating Characteristic (AUROC) down to 0.670. This proves that the data-driven ML tool serves as the mathematical anchor for the architecture.
 - *Removal of the Search Agent (w/o Search):* Disconnecting the web literature parser led to a sharp precision decrease down to 0.338 and dropped system specificity to 0.836.
 - *Removal of the Knowledge Graph Agent (w/o KG):* Removing the path-finding database connection triggered a massive precision drop down to 0.187 and forced the specificity metric down to 0.597.

5.3. Experimental Performance and Pipeline Evaluation: The MADD Architecture

To address the limitations of existing chemical datasets that lack the multi-task complexity of real-world scenarios, the *MADD* framework was evaluated on a new benchmark featuring linked datasets structured around six specific disease cases. The evaluation measured query complexity, orchestrator accuracy, and comparative performance against state-of-the-art baselines:

- **Benchmark Query Complexity Layers:** A high-quality set of user instructions was generated by manually composing 30 realistic inputs from both experts and non-experts (including vague instructions). This was augmented via few-shot prompting across GPT-4o, o1-mini, Claude Sonnet 3.5, and Gemini 1.5 Pro, filtered using

sentence embeddings, and validated by human experts. The final queries were split into three complexity tiers: Dataset S (single-task), Dataset M (1–3 tasks), and Dataset L (4–5 tasks).

- **Orchestrator Accuracy (OA) Results:** On Dataset S, several LLMs were tested on Orchestrator Accuracy (OA), defined as the percentage of correctly selected tools. Using engineered, model-specific optimized system prompts, Llama-3.1-70b achieved the highest performance with a 92.3% OA. Notably, this configuration outperformed o1-mini (67.5%), DeepSeek R1 (88.11%), and Llama3.2-90b (90.7%), establishing Llama-3.1-70b as the default core LLM for MADD.
- **Comparison with Baseline Solutions:** MADD was benchmarked against chemistry-specific language models (LlaSMoL, ChemDFM, X-LoRA-Gemma), the multi-agent ChemAgent, TxGemma, and Phoenix:
 - Standalone LLMs (LlaSMoL, ChemDFM, X-LoRA-Gemma) completely failed to generate structured answers with valid chemical compounds due to a lack of tool access.
 - ChemAgent showed severe pipeline bottlenecks, predicting no more than 2.5% hit molecules that passed the first group of filters (GR1), and made formatting errors in 19.1% of its structured summary outputs, reducing its final system accuracy to 16.4%.
 - MADD avoided these orchestration shortcomings, demonstrating at least 80% final system accuracy on the multi-task datasets. It outperformed ChemAgent by up to 29.1% in tool selection accuracy on Dataset S and Dataset M, while achieving comparable tool selection accuracy on Dataset L.

6. Discussion

The experimental evaluations across these prominent architectures demonstrate the immense value of specialized, multi-agent frameworks over general-purpose systems in fields like digital pharmacology and biochemical modeling. By separating distinct cognitive tasks into specialized sub-units, such as planning high-level ideas, querying structural biomedical knowledge databases, performing live literature web queries, and executing automated code routines, these architectures succeed in bridging the operational gap between abstract text prompts and complex domain requirements.

However, synthesis of the literature reveals several shared limitations that restrict wide-scale deployment of these automated frameworks. A major challenge involves dataset dependency and preprocessing constraints, as systems often assume that users can readily provide well-curated molecular training sets with corresponding target properties. In practice, many researchers lack access to clean, high-quality data repositories, which creates a significant entry barrier. Furthermore, current pipelines require prior knowledge of the biological target or relevant properties, which inherently limits their utility when discovering ligands for fundamentally novel, uncharacterized biological targets.

Scalability and structural rigidity present additional architectural hurdles across state-of-the-art implementations. Most systems still rely heavily on initial human expertise for setup, prompting, and boundary definition, which bottlenecks large-scale automation. The current evaluation benchmarks remain restricted to a limited number of narrow case studies, which are insufficient for a fully comprehensive performance assessment in real-world scenarios. Structurally, the frameworks are bounded by classic baselines rather than the latest chemical engineering techniques, and their documentation remains basic. Finally, mathematical evaluation parameters, such as knowledge graph maximum path limits or structural filtering scores, are often locked to rigid proof-of-concept constants, meaning the intricate trade-offs between computational running costs and complete biological pathway coverage are not yet optimized.

7. Conclusion and Future Work

The integration of Large Language Models into multi-agent systems marks a significant milestone in automating and optimizing drug discovery. By leveraging specialized planners, internal coordinators, and targeted implementation tools, these systems achieve high system accuracy and successfully push the boundaries of virtual screening workflows. They demonstrate robust hit identification efficiency, output highly generalizable chemical scripts, and maintain autonomous discovery capabilities even when applied to unseen disease profiles. This evolution shifts pharmaceutical AI from passive standalone prediction models to active, self-correcting engineering environments.

To resolve current limitations, future work must focus on expanding the autonomy, depth, and integration of the underlying agent networks. On the data side, integrating automated data curation pipelines will eliminate user de-

dependencies, while incorporating specialized modules for hypothesis generation will enable open-ended discovery for novel targets. System optimization requires systematic experimentation to discover the best operational parameters, such as adjusting knowledge graph hop lengths to balance token costs against data coverage, and embedding dedicated retrieval-augmented generation agents to optimize information extraction from private medical repositories.

Additionally, introducing a human-in-the-loop framework will allow laboratory scientists to inject guidance and refine ideas dynamically during long reasoning sessions. Architecturally, future development will expand internal documentation libraries to cover broader biomedical task types, evaluate performance on larger, more diverse datasets, and transition agentic virtual pipelines into physical wet-lab environments by linking software orchestrators directly with automated laboratory hardware.

8. Declaration of Generative AI and AI-assisted technologies in the writing process

During the preparation of this work the author used Gemini 3 in order to correct grammatical errors, improving vocabulary, and rewriting content to establish a more formal and academic tone. After using this tool, the author reviewed and edited the content as needed and take full responsibility for the content of the publication.

References

- [1] Di Tommaso, P., Chatzou, M., Floden, E.W., Barja, P.P., Palumbo, E., Notredame, C., 2017. Nextflow enables reproducible computational workflows. *Nature Biotechnology* 35, 316–319. URL: <http://dx.doi.org/10.1038/nbt.3820>, doi:10.1038/nbt.3820.
- [2] Gilmer, J., Schoenholz, S.S., Riley, P.F., Vinyals, O., Dahl, G.E., 2017. Neural message passing for quantum chemistry, in: *Proceedings of the 34th International Conference on Machine Learning - Volume 70*, JMLR.org. p. 1263–1272.
- [3] Gleeson, M.P., 2008. Generation of a set of simple, interpretable admet rules of thumb. *Journal of Medicinal Chemistry* 51, 817–834. URL: <http://dx.doi.org/10.1021/jm701122q>, doi:10.1021/jm701122q.
- [4] Hou, X., Zhao, Y., Wang, S., Wang, H., 2025. Model context protocol (mcp): Landscape, security threats, and future research directions. URL: <https://arxiv.org/abs/2503.23278>, doi:10.48550/ARXIV.2503.23278.
- [5] Huang, K., Fu, T., Glass, L.M., Zitnik, M., Xiao, C., Sun, J., 2020. Deeppurpose: a deep learning library for drug–target interaction prediction. *Bioinformatics* 36, 5545–5547. URL: <http://dx.doi.org/10.1093/bioinformatics/btaa1005>, doi:10.1093/bioinformatics/btaa1005.
- [6] Inoue, Y., Song, T., Wang, X., Luna, A., Fu, T., 2025. DruGagent: Multi-Agent large language Model-Based reasoning for Drug-Target interaction prediction. *ArXiv*.
- [7] Jumper, J., Evans, R., Pritzel, A., Green, T., Figurnov, M., Ronneberger, O., Tunyasuvunakool, K., Bates, R., Žídek, A., Potapenko, A., Bridgland, A., Meyer, C., Kohl, S.A.A., Ballard, A.J., Cowie, A., Romera-Paredes, B., Nikolov, S., Jain, R., Adler, J., Back, T., Petersen, S., Reiman, D., Clancy, E., Zielinski, M., Steinegger, M., Pacholska, M., Berghammer, T., Bodenstein, S., Silver, D., Vinyals, O., Senior, A.W., Kavukcuoglu, K., Kohli, P., Hassabis, D., 2021. Highly accurate protein structure prediction with alphafold. *Nature* 596, 583–589. URL: <http://dx.doi.org/10.1038/s41586-021-03819-2>, doi:10.1038/s41586-021-03819-2.
- [8] Kim, S., Chen, J., Cheng, T., Gindulyte, A., He, J., He, S., Li, Q., Shoemaker, B.A., Thiessen, P.A., Yu, B., Zaslavsky, L., Zhang, J., Bolton, E.E., 2024. Pubchem 2025 update. *Nucleic Acids Research* 53, D1516–D1525. URL: <http://dx.doi.org/10.1093/nar/gkai059>, doi:10.1093/nar/gkai059.
- [9] Liu, S., Lu, Y., Chen, S., Hu, X., Zhao, J., Lu, Y., Zhao, Y., 2024. Drugagent: Automating ai-aided drug discovery programming through llm multi-agent collaboration. URL: <https://arxiv.org/abs/2411.15692>, doi:10.48550/ARXIV.2411.15692.
- [10] Milacic, M., Beavers, D., Conley, P., Gong, C., Gillespie, M., Griss, J., Haw, R., Jassal, B., Matthews, L., May, B., Petryszak, R., Ragueneau, E., Rothfels, K., Sevilla, C., Shamovsky, V., Stephan, R., Tiwari, K., Varusai, T., Weiser, J., Wright, A., Wu, G., Stein, L., Hermjakob, H., D'Eustachio, P., 2023. The reactome pathway knowledgebase 2024. *Nucleic Acids Research* 52, D672–D678. URL: <http://dx.doi.org/10.1093/nar/gkad1025>, doi:10.1093/nar/gkad1025.
- [11] Seal, S., Huynh, D.L., Chelbi, M., Khosravi, S., Kumar, A., Thieme, M., Wilks, I., Davies, M., Mustali, J., Sun, Y., Edwards, N., Boiko, D., Tyrin, A., Selinger, D.W., Parikh, A., Vijayan, R., Kasbekar, S., Reid, D., Bender, A., Spjuth, O., 2025. Ai agents in drug discovery. URL: <https://arxiv.org/abs/2510.27130>, doi:10.48550/ARXIV.2510.27130.
- [12] Solovov, G.V., Zhidkovskaya, A.B., Orlova, A., Gubina, N., Vepreva, A., Golovinskii, R., Tonkii, I., Dubrovsky, I., Gurev, I., Gilemkanov, D., Chistiakov, D., Aliev, T.A., Poddikov, I., Zubkova, G., Skorb, E.V., Vinogradov, V., Boukhanovsky, A., Nikitin, N., Dmitrenko, A., Kalyuzhnaya, A., Savchenko, A., 2025. MADD: Multi-agent drug discovery orchestra, in: *Christodouloulopoulos, C., Chakraborty, T., Rose, C., Peng, V. (Eds.), Findings of the Association for Computational Linguistics: EMNLP 2025*, Association for Computational Linguistics, Suzhou, China. pp. 6956–6998. URL: <https://aclanthology.org/2025.findings-emnlp.367/>, doi:10.18653/v1/2025.findings-emnlp.367.
- [13] Szklarczyk, D., Nastou, K., Koutrouli, M., Kirsch, R., Mehryary, F., Hachilif, R., Hu, D., Peluso, M.E., Huang, Q., Fang, T., Doncheva, N.T., Pyysalo, S., Bork, P., Jensen, L.J., von Mering, C., 2024. The string database in 2025: protein networks with directionality of regulation. *Nucleic Acids Research* 53, D730–D737. URL: <http://dx.doi.org/10.1093/nar/gkai113>, doi:10.1093/nar/gkai113.

- [14] Tropsha, A., Isayev, O., Varnek, A., Schneider, G., Cherkasov, A., 2023. Integrating qsar modelling and deep learning in drug discovery: the emergence of deep qsar. *Nature Reviews Drug Discovery* 23, 141–155. URL: <http://dx.doi.org/10.1038/s41573-023-00832-0>, doi:10.1038/s41573-023-00832-0.
- [15] Wang, J., Yi, X., Guo, R., Jin, H., Xu, P., Li, S., Wang, X., Guo, X., Li, C., Xu, X., Yu, K., Yuan, Y., Zou, Y., Long, J., Cai, Y., Li, Z., Zhang, Z., Mo, Y., Gu, J., Jiang, R., Wei, Y., Xie, C., 2021. Milvus: A purpose-built vector data management system, in: *Proceedings of the 2021 International Conference on Management of Data*, ACM. p. 2614–2627. URL: <http://dx.doi.org/10.1145/3448016.3457550>, doi:10.1145/3448016.3457550.
- [16] Wei, J., Wang, X., Schuurmans, D., Bosma, M., Ichter, B., Xia, F., Chi, E., Le, Q., Zhou, D., 2022. Chain-of-thought prompting elicits reasoning in large language models [arXiv:2201.11903](https://arxiv.org/abs/2201.11903).
- [17] Wu, Q., Bansal, G., Zhang, J., Wu, Y., Li, B., Zhu, E., Jiang, L., Zhang, X., Zhang, S., Liu, J., Awadallah, A.H., White, R.W., Burger, D., Wang, C., 2023. AutoGen: Enabling Next-Gen LLM applications via Multi-Agent conversation [arXiv:2308.08155](https://arxiv.org/abs/2308.08155).
- [18] Yao, S., Zhao, J., Yu, D., Du, N., Shafran, I., Narasimhan, K., Cao, Y., 2022. ReAct: Synergizing reasoning and acting in language models [arXiv:2210.03629](https://arxiv.org/abs/2210.03629).
- [19] Zdrazil, B., Felix, E., Hunter, F., Manners, E.J., Blackshaw, J., Corbett, S., de Veij, M., Ioannidis, H., Lopez, D.M., Mosquera, J.F., Magarinos, M.P., Bosc, N., Arcila, R., Kizilören, T., Gaulton, A., Bento, A.P., Adasme, M.F., Monecke, P., Landrum, G.A., Leach, A.R., 2023. The chembl database in 2023: a drug discovery platform spanning multiple bioactivity data types and time periods. *Nucleic Acids Research* 52, D1180–D1192. URL: <http://dx.doi.org/10.1093/nar/gkad1004>, doi:10.1093/nar/gkad1004.